

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

TEAM NAME: Its Works On My Machine

TEAM CODE: 106

TEAM MEMBERS	EMAIL
EDISON KONG BANG JEAN (LEADER)	edisonkong05@gmail.com
DARREN WONG CHEE LIANG	darren05wong@gmail.com
GOH JUN TECK	teck0423@gmail.com
LIU YOU ZHENG	liuyz8715@gmail.com
LIEW TIAN EN	ahenahyi05052004@gmail.com

1. Introduction:

SynapseCore is an AI-powered agentic workflow platform for multinational or multi-branch companies. It solves the “messy information” problem where branch offices submit unstructured operational issues, reports, requests, or execution updates, and headquarters needs to convert them into structured, validated, trackable projects.

The system acts as a coordination layer between Branch Offices and HQ. Branch users submit operational signals, while the AI workflow engine analyzes the input, identifies project opportunities, generates phase plans, validates groundedness, recommends approvals, and reviews execution outcomes.

1.1. Purpose:

This SAD explains the technical scope, system design, AI service integration, data flow, database model, and delivery plan for SynapseCore. It acts as a reference for developers, testers, and stakeholders to understand how the frontend, backend API routes, AI workflow engine, mock/real runtime modes, and persistence layer interact.

1. Architecture:

A cloud-native platform utilizing an agentic service layer. Services are decoupled to allow the Large Language Models, LLM reasoning engine to operate independently from the frontend and data persistence layers.

2. Data Flows:

Details how unstructured "messy" data moves from the Branch Manager's portal, through the GLM reasoning engine, into a structured PostgreSQL schema, and finally populates the HQ dashboard.

3. Model Process:

The SynapseCore System processes unstructured inputs from Branch Offices through an AI-driven workflow pipeline. The system first analyzes and converts messy input into structured data, then validates the output to ensure accuracy. Based on this, it creates projects and generates phase-based plans. Each plan is validated before execution. After execution, the system re-analyzes the outcome and either generates the next phase or closes the project. This creates a continuous loop:

Input → Analyze → Validate → Project → Plan → Execute → Evaluate → Next Phase / Close → Report

The process is stateful and adaptive, with AI acting as the core reasoning engine throughout all stages.

4. Role of Reference:

This SAD serves as a technical reference for developers, testers, and stakeholders. It explains the system architecture, AI workflow, data flow, and component interactions, helping all parties understand how the SynapseCore System is designed, implemented, and operates. It also supports development, testing, and future system improvements.

1.2. Background:

In many MNCs, branch offices report issues through emails, chats, forms, or informal documents. HQ must manually understand the issue, decide whether it becomes a project, request missing evidence, approve actions, and monitor outcomes. This causes slow response time, inconsistent decision quality, and poor traceability. SynapseCore centralizes this process by using AI to structure branch inputs into actionable workflows while keeping human validation and HQ approval in the loop.

1) Previous Version:

- a. Branch offices submitted issues, requests, and reports through multiple channels (email, chat, spreadsheets, documents), while HQ handled everything manually—from reviewing cases to identifying problems and deciding actions.
- b. Project creation, approvals, and follow-ups were managed separately on a case-by-case basis, often leading to missing information discovered late.
- c. Project phases, execution updates, and reports were not connected, making tracking inconsistent.
- d. HQ had limited visibility across branches, resulting in a slower, less consistent, and harder-to-audit process.

2) Changes in Major Architectural Components

- a. Backend APIs now manage all core functions, including inputs, projects, phases, approvals, reports, authentication, and AI requests.
- b. A dedicated AI service layer was added, using structured prompts, validation checks, retry handling, and fallback responses.
- c. Data was reorganized into structured entities (projects, phases, plans, validations, approvals, execution updates, and reports), with role-based access control for HQ and branch users.

3) New capabilities that are only introduced in this

- a. AI can convert unstructured branch inputs into structured insights, identifying issue type, urgency, risks, opportunities, missing data, and confidence levels.
- b. The system can recommend project creation, generate phase-based plans, and validate them for completeness or risks.

- c. HQ receives AI-supported approval recommendations, while branches can submit execution updates for each phase.
- d. The platform tracks execution outcomes, suggests next steps, and provides dashboards and reports for better visibility into progress, risks, approvals, and validation issues.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Branch Office	Summit messy operational inputs, requests and phase execution updates	Simple inputs flow, clear project status, visible next actions
Headquarter	Review projects, approve decisions, monitoring risk and branch performance	Reliable structured summaries, validation warnings, approval control

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based HQ and Branch workflow platform
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS
Backend	Next.js API route handlers under app/api
AI Layer	Structured AI service layer using Z.AI/GLM-compatible chat completions, with deterministic mock fallback
Database	Current implementation uses in-memory/mock store and JSON system database; future real mode supports external database through DATABASE_URL

2.1.2 LLM as Service Layer

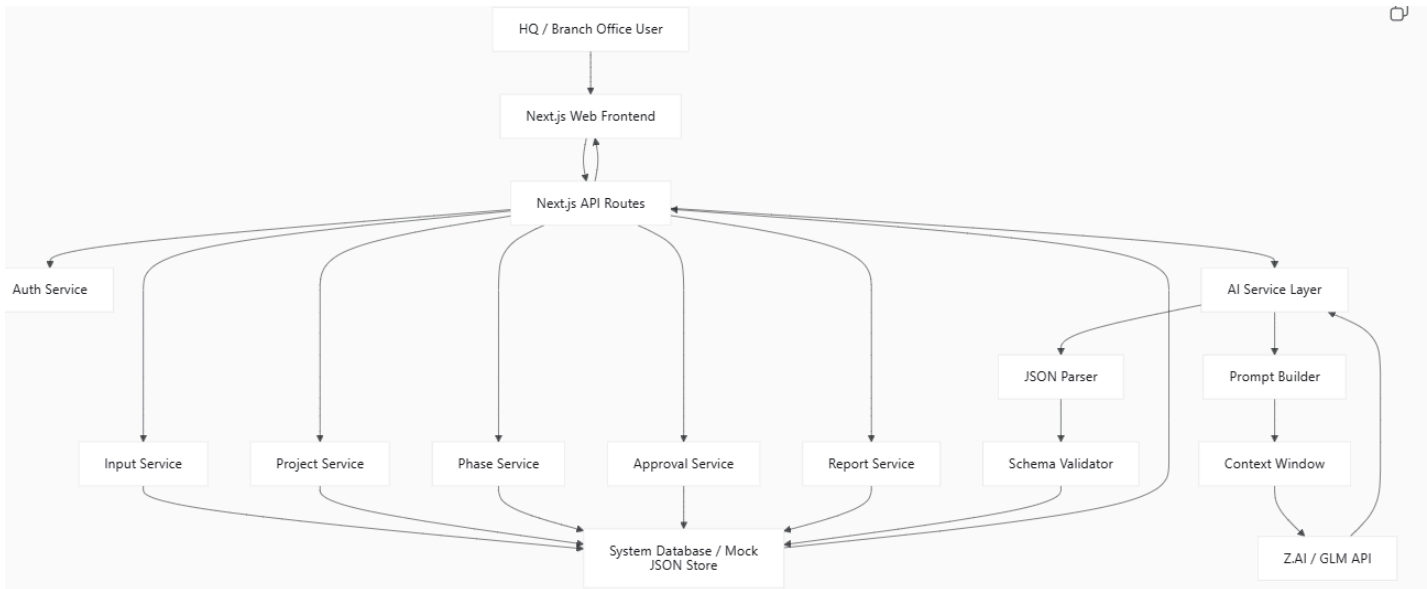
SynapseCore does not treat AI as a generic chatbot. Instead, the GLM is implemented as a structured service layer that powers the system's workflow engine.

The AI operates through a pipeline of modular components:

- Prompt Builder: Constructs structured prompts based on workflow stage, user role, and input data
- Context Manager: Selects relevant project, phase, and input data to control the AI context
- AI Execution (GLM API): Sends requests to the LLM for reasoning, planning, and analysis
- Response Parser: Converts AI output into structured JSON format
- Validation Layer: Ensures output correctness, completeness, and schema compliance
- Retry & Fallback Mechanism: Re-runs or switches to a deterministic pipeline if validation fails

This design ensures that AI outputs are reliable, structured, and directly usable within the system, enabling consistent multi-step workflow execution instead of simple conversational responses.

2.1.3 Dependency Diagram



a. How the prompts are being constructed and sent to the GLM API

i. Task Selection: The backend identifies the specific objective required (e.g., analyzing branch input, generating phase plans, or validating an approval decision).

ii. Dynamic Template Filling: The system selects a pre-defined prompt template based on the task. It injects real-time data such as branch reports, project status, and business constraints into the template.

iii. Constraint Enforcement: Every prompt explicitly instructs the GLM to act as a specific persona (e.g., "Operations Manager") and mandates a structured JSON-only response. This prevents conversational "noise" and ensures the output is machine-readable.

iv. API Execution: The final package (System Instructions + User Data) is sent to the Z.AI GLM API via an asynchronous Chat Completion request.

b. What goes into the context window?

The context window contains only the information needed for the current AI task. It may include:

- i. Current workflow stage
- ii. User role, either HQ or Branch Office
- iii. Raw input submitted by the branch
- iv. Source type, such as manual form, branch report, feedback, uploaded document, or outcome update
- v. Current project information
- vi. Current phase information
- vii. Existing AI analysis
- viii. Existing plan
- ix. Validation result
- x. Execution update
- xi. Required JSON response schema

c. How is your system receiving, parsing the responses and passing to the next system component?

- The backend expects the response to be in JSON format.
- The system then parses the JSON response.
- After parsing, the response is validated.
- The validation layer checks:
 - i. Whether the response is valid JSON
 - ii. Whether the correct root key is used
 - iii. Whether all required fields are included
 - iv. Whether the values follow the correct format
 - v. Whether confidence scores are within the correct range
 - vi. Whether unsupported or extra fields are included
- If the response is valid, the result is saved into the system database.
- The saved result is then passed to the next system component.
- Where the limitation of token are enforced or input are chunked before it actually reaches to GLM

d. Major API Calls

Frontend Action	API Endpoint	Main Result
Login	POST /api/auth/login	Creates demo session
Submit input	POST /api/inputs	Stores input and runs AI analysis
Analyze input	POST /api/ai/analyze-input	Generates structured ai_analysis
Create project	POST /api/ai/create-project-from-input	Creates project and initial phase
Generate plan	POST /api/ai/generate-phase-plan	Creates phase plan
Validate plan	POST /api/ai/validate-phase-plan	Stores validation result
Request approval	POST /api/approvals/request	Creates HQ approval request
Decide approval	POST /api/approvals/[id]/decision	Updates approval/project/phase state
Execute update	POST /api/phases/[id]/execute-update	Stores execution result and triggers review
Reports	GET /api/reports/*	Returns project, branch, risk, or validation reports

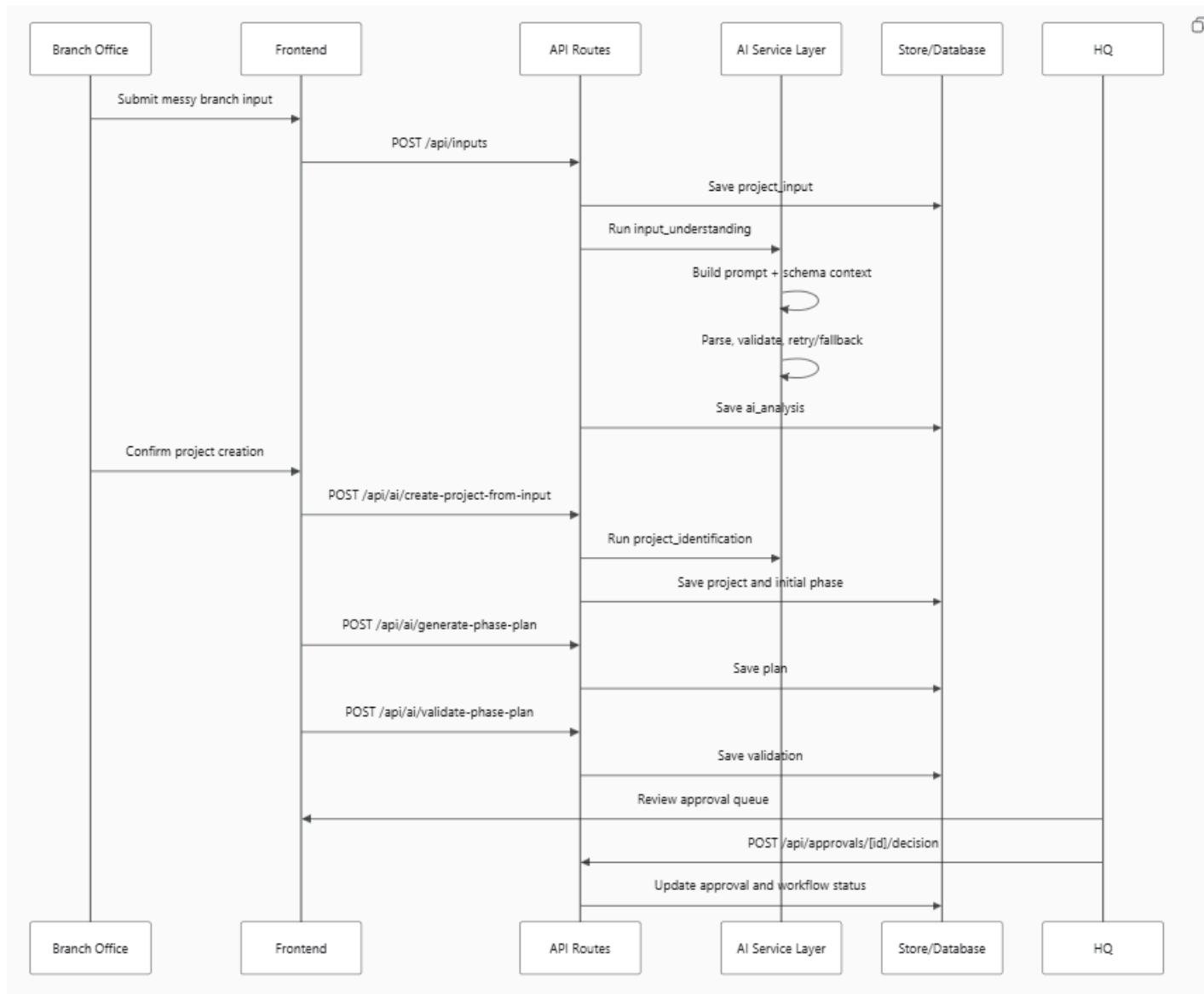
- e. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM

For MVP, the system limits AI context by sending only the relevant project, phase, validation, execution update, and selected input data instead of the entire database. Future enhancement should add explicit token counting, file chunking, summarization, and caching for repeated branch/project contexts.

- f. Do show how frontend, backend, services, database and external API interact with each other.
- HQ and Branch Office users interact with the system through the Next.js frontend.
 - The frontend sends requests to backend API routes.
 - API routes call the correct backend service depending on the action.
 - Backend services read from and write to the system database or mock JSON store.
 - When AI reasoning is needed, the backend service calls the AI Service Layer.
 - The AI Service Layer builds the prompt and sends it to the GLM API.
 - GLM returns a structured JSON response.
 - The backend validates the response before storing it.
 - The frontend receives clean structured data from the backend and displays it in pages such as dashboard, projects, approvals, validation center, and reports.
- g. Must highlights the distributed components if the system is planning to be deployed across multiple services

Component	Current MVP	Future Deployment
Frontend	Next.js web app	Hosted on Vercel, AWS, or static frontend hosting
Backend API	Next.js API routes	Can be separated into backend services
AI Service Layer	Internal service module	Can become independent AI microservice
Database	Mock/JSON store	PostgreSQL database
GLM API	External AI API	External Z.AI/GLM service
Reports	Internal report service	Can become analytics/reporting service
Monitoring	Basic application behavior	Logs, metrics, alerts, rollback system

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend

- [Next.js](#) - Provides the main web application structure, routing, pages, layouts, and connection between frontend screens and backend API routes.
- React - Builds reusable UI components such as dashboards, project pages, approval pages, forms, tables, and reports.
- TypeScript - Adds type safety for frontend data, API responses, project records, AI outputs, and shared system models.
- Tailwind CSS - Styles the user interface using utility classes, including layout, spacing, colors, buttons, cards, tables, and responsive design.

2.2.2. Backend

- [Next.js](#) & API Routes - Handles backend endpoints such as login, input submission, project creation, AI workflow calls, approvals, reports, and dashboard data.
- TypeScript - Ensures backend services, request payloads, response objects, and database records follow consistent data types.
- Modular Service Layer - Separates system logic into services such as Input Service, Project Service, Phase Service, Approval Service, Report Service, and AI Service.

2.2.3. Database

- Mock JSON Store (local JSON-backed persistence) - Used in the hackathon MVP to store demo data such as offices, workflows, projects, requests, issues, and company database records.

2.2.4. Cloud/Deployment

FUTURE DEPLOYMENT (PRELIMINARY ROAD NOT COMPULSORY)

- AWS EC2 - Can host the application server for more controlled production deployment.
- AWS RDS PostgreSQL - Can host the production PostgreSQL database.
- AWS S3 - Can store uploaded documents, static assets, logs, and supporting evidence files.

2.3. Key Data Flows

- 2.3.1. Here, do show the how data are moving through the system and how its structure and stored
- 2.3.2. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases
- 2.3.3. Show how proper laws are met (e.g. Black Hole)



2.3.4. Normalized Database Schema

Entity	Purpose
branches	Stores branch identity and region
users	Stores HQ and Branch Office users
project_inputs	Stores raw submitted input and source type
ai_analysis	Stores structured AI interpretation of each input
projects	Stores main project records
phases	Stores ordered project phases
plans	Stores AI-generated phase plans
validations	Stores groundedness and human-review checks
execution_updates	Stores branch execution evidence and outcomes
approvals	Stores HQ approval requests and decisions

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Feature	Description
1	Branch Input Submission	Branch Office users can submit unstructured issues, requests, reports, or evidence through the platform.
2	AI Input Analysis	The system uses GLM to analyze the branch input and extract issue type, urgency, summary, risks, opportunities, missing information, and confidence score.
3	Workflow & Project Creation	The system can convert validated branch input into a structured project with project title, summary, priority, owner role, and initial phase.
4	HQ Approval	HQ users can review approval requests and decide to approve, reject, request revision, or escalate.
5	Reports and Dashboard	HQ can view project reports, branch reports, risk reports, validation reports, and dashboard summaries.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirement	Implementation
Scalability	Support multiple branches and workflows	Modular API routes and service modules
Reliability	AI failures must not break demo flow	Mock fallback when API/config/validation fails
Maintainability	AI, services, types, and UI are separated	src/modules, src/services, types, components structure
Security	Branch Office can only access own branch data	Role-based checks in services
AI Safety	Unsupported AI output must not proceed silently	Strict JSON schema validation and human-review gating

3.3. Out of Scope / Future Enhancements

These are the high-level features planned for the future “Grand Vision” of SynapseCore, but they are not included in the UMHackathon 2026 MVP:

- a. Real-time in-app chat between HQ users and Branch Office users.
- b. Real-time notification system for new approvals, rejected requests, revised plans, and urgent escalations.
- c. Mobile application version for Branch Office users.
- d. Automated email or WhatsApp reminders for pending approvals and execution updates.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

New AI workflow features can first be released to a small group of internal HQ users, such as 5% of users. The team will monitor AI response accuracy, validation failure rate, approval errors, user feedback, and system errors. If no major issues are detected, the feature can gradually roll out to more users, such as 25%, 50%, and finally 100%.

4.1.2. Emergency Rollback (ER) & Golden Release:

For future deployment, SynapseCore should keep a stable “Golden Release” version of the application. If the latest version causes serious errors, the system can rollback to the Golden Release. Emergency rollback should be triggered when critical issues happen, such as:

- AI response failure rate becomes too high
- Users cannot submit branch inputs
- HQ approval decisions cannot be saved

4.1.3. Priority Matrix:

Priority	Severity	Trigger Condition	Action Taken
P1	Critical	Branch users can access other branch data, HQ approval decisions cannot be saved, or the system is completely down	Immediate emergency rollback to Golden Release, disable affected feature, investigate root cause, restore service as soon as possible
P2	High	GLM API fails repeatedly, AI analysis cannot be generated, project creation fails, or validation results are incorrect for many users	Switch to mock AI fallback, pause real AI mode, fix AI/service issue, redeploy after testing
P3	Medium	Reports or dashboards show delayed or partially incorrect data, approval queue is slow, or some project status updates are not refreshed properly	Log the issue, patch the affected service, verify data consistency, release a minor fix

P4	Low	UI layout issues, typo, small display bug, or non-critical page loading delay	Add to backlog, fix in the next sprint or next minor release
----	-----	---	--

4.2. Monitoring:

Track API errors, AI fallback count, validation warnings, approval backlog, and project status changes

4.3. Assumptions:

Users have stable internet, branch staff can provide sufficient evidence, HQ remains final decision maker

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Acts as the core AI reasoning engine. It analyzes messy branch input, generates structured AI analysis, creates project recommendations, generates phase plans, validates plans, recommends approvals, and reviews execution updates	High risk. If the GLM API is down, rate-limited, slow, or returns invalid output, the core AI workflow may be affected. Mitigation: use mock AI fallback, retry logic, strict JSON validation, and graceful error messages.
Hosting Platform, Vercel or AWS	Hosts the SynapseCore web application so HQ and Branch Office users can access it online.	Medium risk. If hosting service has downtime, users cannot access the platform. Mitigation: keep a Golden Release and prepare rollback deployment.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day	Activity
Day 1-2	Requirement analysis, workflow definition, SAD and architecture planning
Day 3-4	Frontend layout, dashboard, branch/HQ navigation
Day 5-6	Backend API routes, mock data store, role access logic
Day 7-8	AI prompt pipeline, schema validation, fallback responses
Day 9	Reports, approvals, demo data polishing
Day 10	Testing, bug fixing, final demo preparation

5.2. Team Members Role: Highlight the role of team members

Member	Role
Edison Kong Bang Jean	Team lead, architecture coordination, presentation
Darren Wong Chee Liang	Frontend and UI implementation
Goh Jun Teck	Backend API and data model
Liu You Zheng	AI workflow, prompt design, validation logic
Liew Tian En	QA, documentation, testing, demo preparation

5.3. Recommendations:

For production, SynapseCore should add PostgreSQL persistence, Redis caching for repeated AI contexts, structured audit logs, OpenTelemetry monitoring, explicit token-budget enforcement, and a queue-based AI execution layer for long-running workflows.